



dsi | Departamento de
Sistemas e Informática

Generador de formas de ondas arbitrarias (AWG) con RPI Pico

Facultad de Ciencias Exactas, Ingeniería y Agrimensura
(UNR-FCEIA)

08 / 25

Índice

- ① Introducción
- ② Firmware
 - POO
 - Máquina de estados
 - Sistema Operativo en tiempo real
- ③ Hardware
- ④ Ensayos y video demostrativo

Introducción

- *¿Porqué un generador de funciones? ¿De donde nació la idea?*
- *¿Cuáles fueron las motivaciones?*
- *¿Qué es un AWG? ¿Porqué la elección de ese micro en particular?*

Introducción

Un generador de formas de onda arbitrarias (AWG) permite generar funciones periódicas cargando puntos en un buffer y reproduciéndolos mediante un DAC a la frecuencia solicitada. La resolución depende de la profundidad del buffer y del DAC, afectando la frecuencia máxima alcanzable. Mayor resolución implicará menor frecuencia máxima.

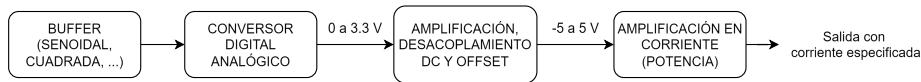


Figura: Diagrama en bloques del AWG

Raspberry Pi Pico

- **Microcontrolador:** RP2040, ARM Cortex-M0+ dual-core a 133 MHz.
- **Memoria:** 264 KB de SRAM y 2 MB de flash externa (QSPI).
- **Periféricos:** 2x I2C, 2x SPI, 2x UART, 16 canales PWM, ADC de 12 bits (3 canales + temperatura interna).
- **PIO:** 2 bloques con 4 máquinas de estado programables cada uno.
- **GPIO:** 26 pines (3.3 V, multifunción, no toleran 5 V).
- **USB:** 1.1, modo host o dispositivo.
- **Alimentación:** 1.8–5.5 V (regulador interno a 3.3 V).
- **Programación:** C/C++, MicroPython, Arduino.

Programmable Input/Output (PIO) y DMA

PIO (Programmable I/O): es un periférico programable que permite implementar protocolos personalizados o de alta velocidad directamente en hardware, sin cargar la CPU. Posee 2 bloques con 4 máquinas de estado cada uno. Es ideal para bit-banging preciso o interfaces como WS2812, I2S o DVI. En este caso fue utilizado para generar la señal de salida en la escalera R-2R.

DMA (Direct Memory Access): transfiere datos entre memoria y periféricos sin intervención del CPU, liberando carga de procesamiento. Cuenta con 12 canales independientes y se usa comúnmente con PIO, SPI o ADC para transferencias continuas de datos. En nuestro caso lo utilizamos para transferir el buffer en RAM al PIO.

Programación orientada a objetos

¿Cuál fue el enfoque de la programación?

El firmware fue escrito basado en el concepto de la programación orientada a objetos. Para esto mismo cada archivo/objeto representa una porción específica del hardware. En especial pueden destacarse las etapas de **amplificación, desacoplamiento DC y filtros PB** (offset). Además se hizo énfasis en una organización en forma de capas, con la idea de que las capas asociadas a niveles superiores representen una mayor interacción con el usuario, mientras que capas de niveles inferiores una mayor abstracción para este.

Programación orientada a objetos

Ej: el usuario configura a través de una pantalla un offset de +2 V.

¿Cómo dicho se refleja en el hardware?

La información solicitada por el usuario genera un evento que es capturado por la *máquina de estados*. Esta se encarga de transferir los datos al objeto *AWG*, responsable de generar la señal deseada. El *AWG* interactúa con el objeto *offset*, el cual contiene internamente un objeto denominado *filtro RC*. Dicho filtro produce una señal PWM cuyo ciclo de trabajo se ajusta en función de los datos recibidos. Esta ingresa a un AO en configuración de filtro pasa bajo, produciendo una señal continua positiva. En la figura 2.1 puede verse el esquema.

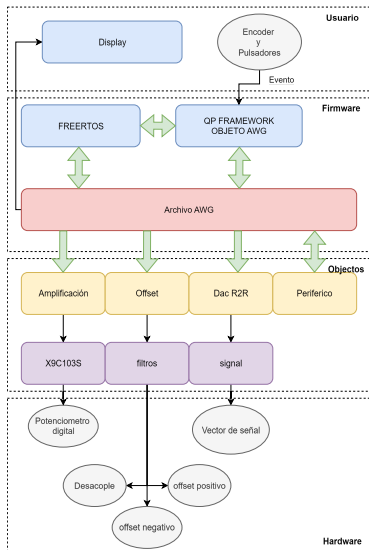


Figura: Estructura en capas del firmware.

Máquina de estados

Con la intención de generar un menú de configuración para los distintos parámetros de la señal como *amplitud*, *frecuencia*, *offset* y *tipo de señal*, se diseñó una máquina de estados. Utilizamos el framework de Quantum Leaps, en el que desarrollamos de forma gráfica los distintos estados, eventos y temporizadores a utilizar. Podemos ver una representación de esta máquina de estados en la figura 2.2.

Máquina de estados

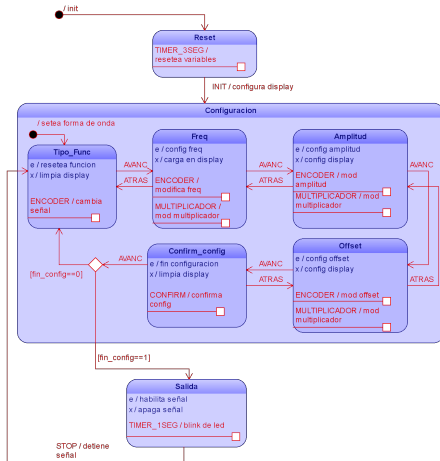


Figura: Máquina de estados.

Dentro de los estados más importantes podemos destacar los siguientes:

- **Tipo_func:** Selecciona el tipo de señal de salida.
- **Freq:** Configura la frecuencia de salida.
- **Amplitud:** Configura la amplitud se la señal final.
- **Offset:** Configura el offset de la salida.

Mediante un encoder rotativo que se encuentra en la PCB, se permite variar cada uno de los parámetros antes mencionados. A su vez con los pulsadores permite al usuario moverse entre las distintas configuraciones.



Figura: Encoder rotativo.

Sistema Operativo en Tiempo Real

- *¿Qué es un RTOS? ¿Porqué utilizamos FreeRTOS?*
- *¿Cómo están distribuidas las tareas?*
- *¿Cómo se relaciona con la MEF de antes?*

Para la gestión del firmware se utilizó FreeRTOS, elegido por ser de código abierto y capaz de integrarse con Quantum Leaps. La máquina de estado se ejecuta como una tarea estática, mientras que en paralelo corren otras tareas esenciales: detección de pulsadores, manejo de interrupciones del encoder y generación de la señal (AWG) que son dinámicas.

Hardware

El hardware se compone de varias etapas que permiten producir una señal de acuerdo a las especificaciones del usuario. Se mencionan las siguientes:

- **Conversión analógica digital**
- **Amplificación**
- **Desacople DC**
- **Offset de continua**

¿Qué hace a grandes rasgos cada etapa? ¿En qué radica su importancia?

Ensayos realizados

Senoidal:

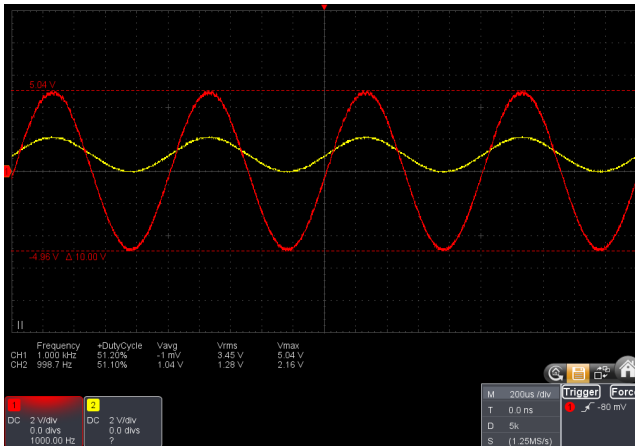


Figura: 10 V_{pp} y $f = 10$ KHz.

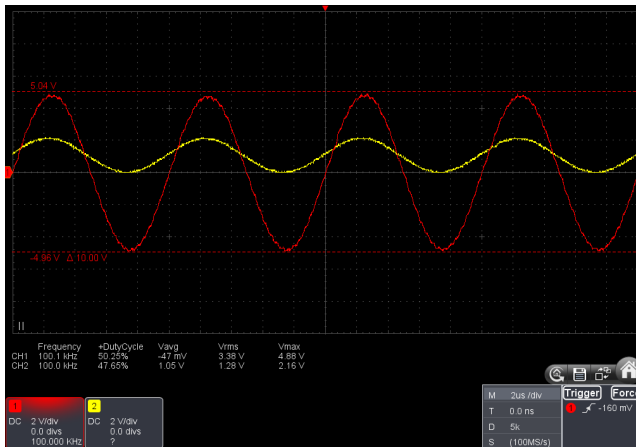


Figura: 10 V_{pp} y $f = 100$ KHz.

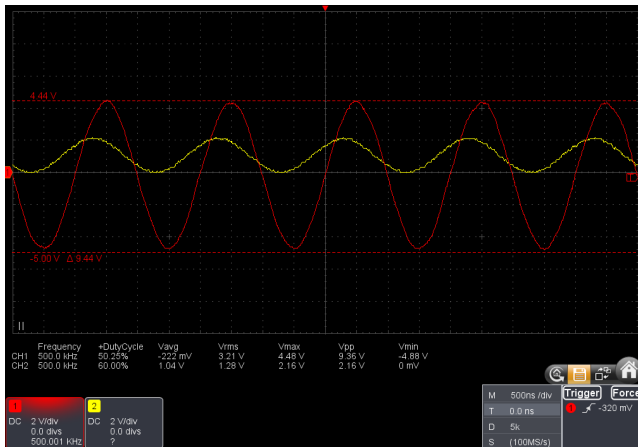


Figura: 10 V_{pp} y $f = 500$ KHz.

Cuadrada:

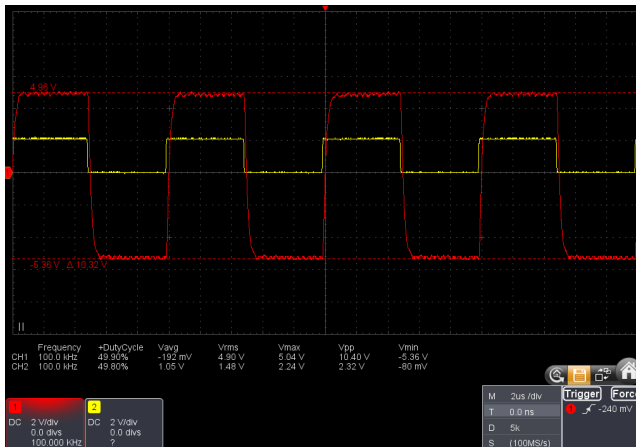


Figura: 10 V_{pp} y $f = 100$ KHz.

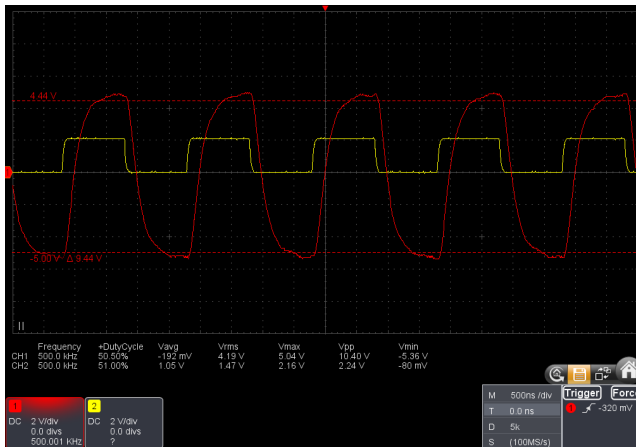
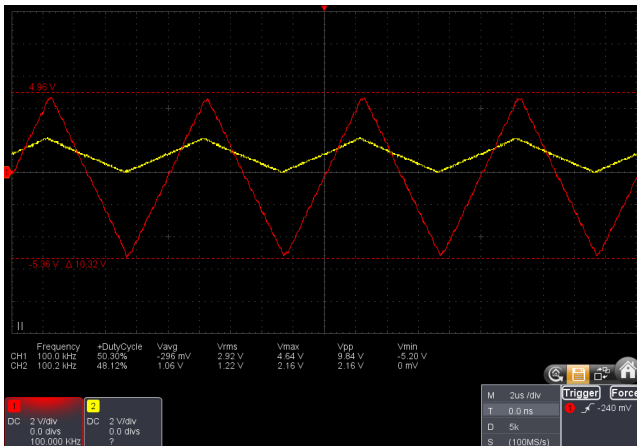
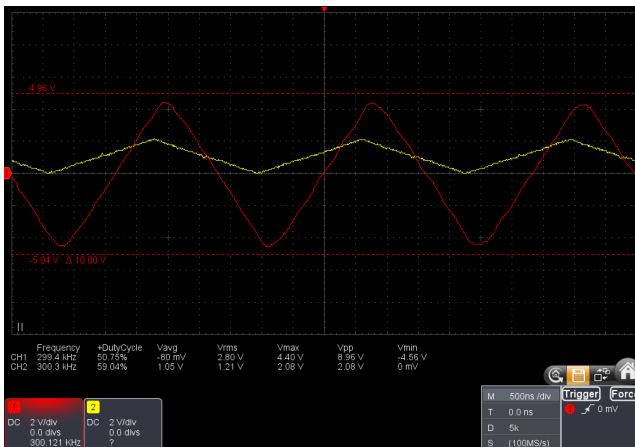


Figura: 10 V_{pp} y $f = 500$ KHz.

Triangular:





Video demostrativo



Gracias por mirar y escuchar!